Automating Workflows (make)

Lecture 15

Dr. Colin Rundel

Relative and absolute paths

An absolute path starts at the root of the file system and specifies one location.

```
/Users/colin/project/data/raw.csv
```

A relative path starts from the working directory (usually where the process was started).

Path	Meaning
`data/raw.csv`	`raw.csv` inside `data`, inside the working directory
`./clean_data.R`	`clean_data.R` in the working directory
`../other/x.csv`	up one directory, then into `other`

Absolute paths break when a project moves or is copied to another computer.

For reproducibility, never use absolute paths. Instead use relative paths that assume the project root is the working directory.

If the first line of your R script is

```
setwd("C:\Users\jenny\path\that\only\I\have")
```

I* will come into your office and
SET YOUR COMPUTER ON FIRE 🔥.

* or maybe Timothée Poisot will

Automating a project

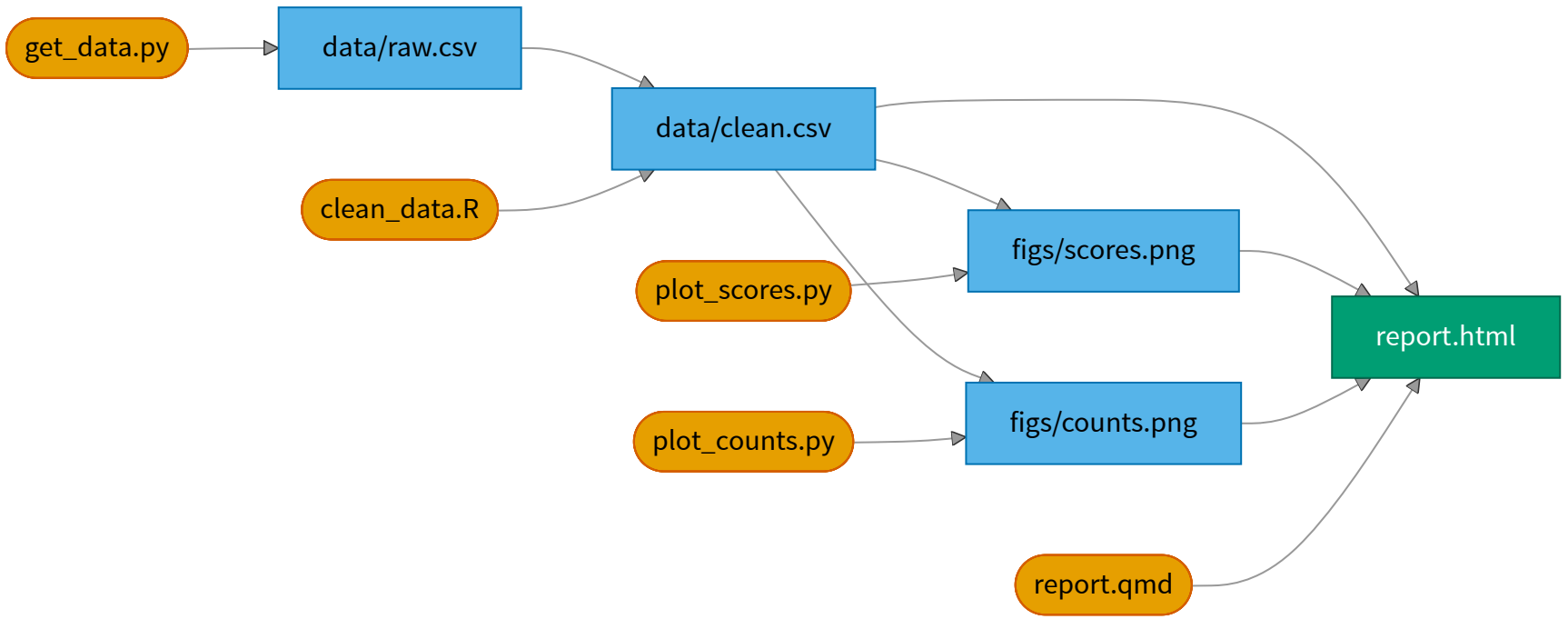
A typical report

A data analysis often involves several scripts that read files and write results for the next step. This project combines Python, R, and Quarto to produce an HTML report.

```
project/
├── get_data.py       # downloads the raw data to data/raw.csv
├── clean_data.R      # cleans it into data/clean.csv
├── plot_scores.py    # draws figs/scores.png from the clean data
├── plot_counts.py    # draws figs/counts.png from the clean data
├── report.qmd        # renders report.html from the clean data and the figures
├── pyproject.toml    # declares the Python dependencies
└── uv.lock           # records their resolved versions
```

Our goal is to keep `report.html` up to date as the scripts and data change, without rerunning steps unnecessarily.

What depends on what?



If we edit `clean_data.R`, which steps must run again to bring the report up to date?

What about `report.qmd`?

`make`

- A build tool that creates files from other files by following a set of rules you write down.
- Originally created by Stuart Feldman in 1976 at Bell Labs to build C programs, but it is language agnostic. Anything that reads files and writes files can be automated with `make`.
- Available on almost any operating system with basic development tools installed.
- The rules (usually) live in a text file named `Makefile` in the project directory.

Example rule - `data/clean.csv`

The dependency diagram shows two inputs for this file: the cleaning script and the raw data. We can express that relationship as a rule using the following syntax:

```
data/clean.csv: clean_data.R data/raw.csv
  mkdir -p data
  Rscript clean_data.R
```

- The *target*, `data/clean.csv`, is the output we want to build.
- The *prerequisites*, `clean_data.R` and `data/raw.csv`, are inputs. Changing either can make the output outdated.
- The *recipe* is the two indented shell commands that produce the output.

A full Makefile

We can express the full dependency graph as follows:

```
report.html: report.qmd data/clean.csv figs/scores.png figs/counts.png
  quarto render report.qmd

figs/scores.png: plot_scores.py data/clean.csv
  mkdir -p figs
  uv run python plot_scores.py

figs/counts.png: plot_counts.py data/clean.csv
  mkdir -p figs
  uv run python plot_counts.py

data/clean.csv: clean_data.R data/raw.csv
  mkdir -p data
  Rscript clean_data.R

data/raw.csv: get_data.py
  mkdir -p data
  uv run python get_data.py
```

Previewing a build

With no target named, `make` selects the first target in the Makefile.

If we invoke `make` with the `-n` flag it previews the build's commands without running them.

```bash
make -n
```

```
mkdir -p data
uv run python get_data.py
mkdir -p data
Rscript clean_data.R
mkdir -p figs
uv run python plot_scores.py
mkdir -p figs
uv run python plot_counts.py
quarto render report.qmd
```

From this we can see that `make` works backward from the report to find its prerequisites, then plans the commands in dependency order.

From preview to build

First run:

```bash
make
```

```
mkdir -p data
uv run python get_data.py
mkdir -p data
Rscript clean_data.R
mkdir -p figs
uv run python plot_scores.py
mkdir -p figs
uv run python plot_counts.py
quarto render report.qmd
```

Second run:

```bash
make
```

```
make: `report.html' is up to date.
```

Third run:

```bash
make
```

```
make: `report.html' is up to date.
```

Requesting an output

Specific targets can be built directly by passing their name as an argument to `make`.

```bash
rm -f data/clean.csv # remove existing file
make data/clean.csv
```

```
mkdir -p data
uv run python get_data.py
mkdir -p data
Rscript clean_data.R
```

How Make decides what to run

For each requested file, Make follows the same procedure:

1. Check its prerequisites, updating any that need rebuilding.
2. Run its recipe if the target is missing or a prerequisite has a newer modification time.
3. Otherwise, reuse the existing target.

Make compares modification times (timestamps), not file contents. Saving a script can trigger a rebuild even if its output would be unchanged.

What reruns after a change?

Once the project is built, Make reruns only the steps downstream of what changed, which answers the questions from the dependency diagram. Each row starts from an up-to-date project, makes one change, and runs `make`.

Change	Steps that run	Steps skipped
Edit `clean_data.R`	Clean, both plots, render	Download
Edit `report.qmd`	Render	Download, clean, both plots
Delete `figs/scores.png`	Plot scores, render	Download, clean, plot counts
Edit `get_data.py`	Everything	None
Nothing	None	Everything

A deleted output is rebuilt even though its inputs did not change, and the steps after it rerun because the new file is newer.

Make only sees declared dependencies

Make does not inspect scripts to discover their inputs and outputs. The `Makefile` must describe those relationships.

- An omitted input can leave an outdated result that Make considers up to date. Include data, helper scripts, and configuration.
- Changes outside the project are invisible. If the file `get_data.py` downloads is updated online, nothing local is newer, so the raw data is not refreshed.
- A recipe that never creates its target runs again each time that missing target is requested.
- An extra output with no rule of its own may not be recreated if it is deleted.

Editing a recipe also does not trigger a rebuild, because `Makefile` is not listed as a prerequisite. Use `make -B report.html` to force a rebuild of the report and all of its prerequisites, including the download.

Tabs, not spaces

A common error is indenting a recipe with spaces. By default, Make requires a tab at the start of each recipe line.

Spaces will produce a `missing separator` error like the following:

```bash
make
```

```
Makefile:2: *** missing separator.  Stop.
```

If your editor converts tabs to spaces, disable that setting for `Makefile` files. Showing whitespace characters can help you spot the difference.

Recipe lines use separate shells

By default, each recipe line runs in a separate shell. Changes to the working directory, shell variables, or an activated environment do not carry over to the next line.

`Rscript` runs in the original directory:

```
preview:
  cd analysis
  Rscript summary.R
```

`Rscript` runs inside `analysis`:

```
preview:
  cd analysis && Rscript summary.R
```

- `&&` runs the second command only if the first succeeds. With `;`, the second command runs even if `cd` fails.
- A trailing `\` continues a recipe onto the next line, keeping it in the same shell.
- On Unix, the default shell is `/bin/sh`. Do not assume your interactive aliases or Bash/Zsh-specific syntax will work.

Phony targets

Sometimes we want a target that is not a file but rather describes an action. These are called phony targets. Common examples are an `all` target that builds everything and a `clean` target that removes generated files.

```
all: report.html

clean:
	rm -f data/clean.csv figs/scores.png figs/counts.png report.html

.PHONY: all clean
```

`.PHONY` tells Make that these names are actions, so it never looks for files called `all` or `clean`.

This `clean` leaves `data/raw.csv` alone since it was downloaded rather than computed. Only remove files the workflow can regenerate.

Useful make flags

As we have seen, `make` accepts flags that change what it builds and how. The following are some of the most useful.

Option	Effect
`-n`	print the recipes that would run without running them
`-B`	rebuild requested targets and prerequisites, ignoring timestamps
`-j N`	run up to `N` recipes in parallel where the graph allows
`-k`	keep going after an error and build what can still be built
`-s`	do not echo commands
`-f file`	use `file` instead of `Makefile`
`-C dir`	change to `dir` before doing anything

Intermediate Make

Variables

Make variables hold text, such as commands or paths, for reuse. `:=` expands any variable references on the right and stores the result when Make reads the file. `$(NAME)` inserts the stored value.

```
PY := uv run python
R  := Rscript

figs/scores.png: plot_scores.py data/clean.csv
  mkdir -p figs
  $(PY) plot_scores.py

data/clean.csv: clean_data.R data/raw.csv
  mkdir -p data
  $(R) clean_data.R

data/raw.csv: get_data.py
  mkdir -p data
  $(PY) get_data.py
```

Make expands recipe variables before the shell runs. Use `$$` to pass a dollar sign to the shell, as in `echo "$$HOME"` to

Automatic variables

Make also supplies automatic variables whose values depend on the rule being run. They let a recipe refer to its target and prerequisites without repeating their names.

Variable	Meaning
`$@`	the target
`$<`	the first prerequisite
`$^`	all prerequisites, without duplicates
`$(@D)`	the directory part of the target

```
data/clean.csv: clean_data.R data/raw.csv
  mkdir -p "$(@D)"
  $(R) "$<"
```

Here `$(@D)` is `data`, `$<` is `clean_data.R`, and `$(R)` is `Rscript` from the previous slide.

Pattern rules

The project's two plotting scripts, `plot_scores.py` and `plot_counts.py`, each write a PNG to `figs/`. Their rules are identical apart from the names.

A single pattern rule covers both. `%` stands for the part of the name that varies, here `scores` or `counts`, and the same text is substituted into the prerequisites.

```
figs/scores.png: plot_scores.py data/clean.cs
  mkdir -p figs
  uv run python plot_scores.py

figs/counts.png: plot_counts.py data/clean.cs
  mkdir -p figs
  uv run python plot_counts.py
```

```
figs/%.png: plot_%.py data/clean.csv
  mkdir -p "$(@D)"
  $(PY) "$<"
```

A pattern rule only applies to targets that are asked for, so both figures still need to be listed as prerequisites of `report.html`.

Wildcards and functions

Make functions use a `$(name arguments)` syntax. Here we find `.qmd` files in the working directory and construct the corresponding `.html` target names.

```
QMD  := $(wildcard *.qmd)
HTML := $(patsubst %.qmd,%.html,$(QMD))

all: $(HTML)

.PHONY: all

%.html: %.qmd
  quarto render "$<"
```

- `wildcard` finds existing `.qmd` files using globbing.
- `patsubst` replaces `.qmd` with `.html` in each name.

This course's website

The following is a simplified version of the Makefile used to generate this course's website:

```
QMD   := $(wildcard static/slides/*.qmd)
HTML  := $(patsubst %.qmd,%.html,$(QMD))
PDF   := $(patsubst %.html,%.pdf,$(HTML))
THEME := static/slides/slides.scss static/slides/_bootstrap-utilities.scss

all: build

build: $(HTML) $(PDF)
  hugo

static/slides/%.html: static/slides/%.qmd $(THEME)
  quarto render "$<"

static/slides/%.pdf: static/slides/%.html
  Rscript -e "renderthis::to_pdf('$<')"

.PHONY: all build
```

Because `build` is phony, `hugo` runs on every `make`, after the required slide updates.

Environments as prerequisites

Python results also depend on package versions. In a uv project, `pyproject.toml` declares the dependencies and `uv run` keeps `.venv` in step with them, but Make does not know that outputs built with older packages are now stale.

An empty marker file connects the two. Make rebuilds it when `pyproject.toml` changes, and every rule that runs Python lists it as a prerequisite, here shown for the figures.

```
.venv/.synced: pyproject.toml
  uv sync
  touch "$@"

figs/%.png: plot_%.py data/clean.csv .venv/.synced
  mkdir -p "$(@D)"
  $(PY) "$<"
```

Failed recipes and partial outputs

A command can write part of its output before failing. Make stops on the error, but normally leaves that file behind. A later run may consider it up to date because its timestamp is new.

Add this special target to the Makefile:

```
.DELETE_ON_ERROR:
```

If a recipe changes its target and exits with an error, Make deletes the target so the next run tries again. This protects against an incomplete `data/raw.csv` after a failed download.

It does not remove other files the command wrote, and the command must report failure with a nonzero exit status.

Beyond Make

Make coordinates commands using file dependencies and timestamps. Tools designed for data analysis extend this approach to other kinds of dependencies and execution environments.

- `targets` for R tracks R objects and declared file targets. It uses hashes of code and data to detect changes and supports parallel execution.
- `Snakemake` uses a Python-based workflow language with explicit inputs and outputs. It adds support for software environments, clusters, and containers.

Make is a useful starting point for projects organized around scripts and files. Consider these tools when you need to track R objects or manage more complex computing environments.

Summary

Takeaways

- A file rule names an output, its prerequisite inputs, and a recipe that builds it. Recipe lines start with a tab.
- Make rebuilds missing or outdated targets using modification times, not file contents.
- Phony targets like `all` and `clean` name tasks. Use `make -n` to preview the commands for a build.
- Variables and pattern rules reduce repetition when several outputs are built in the same way.
- Make only knows the dependencies you declare, so list every input and check both a fresh build and a second run.

Learning more

- GNU make manual
- minimal make, a short introduction for reproducible research
- Software Carpentry - Automation and Make
- targets and the targets user manual
- Snakemake

File-system reference

Path shortcuts

In a relative path, `.` refers to the working directory and `..` moves up one directory, wherever it appears. A leading `~` is a shortcut for your home directory when expanded by the shell or a path function.

```r
fs::path_abs("..")
```

```
/Users/rundel/Desktop/Sta523-Fa26/website/static
```

```r
fs::path_expand("~/Desktop")
```

```
/Users/rundel/Desktop
```

```python
os.path.abspath("..")
```

```
'/Users/rundel/Desktop/Sta523-Fa26/website/static'
```

```python
os.path.expanduser("~/Desktop")
```

```
'/Users/rundel/Desktop'
```

Building and taking apart paths

Use path functions to join directory and file names with the appropriate separator for your operating system.

Other path functions extract the file name, directory, or extension. Python's `splitext()` returns both the path without its extension and the extension, including the dot.

R
```r
file.path("data", "penguins.csv")
```
```
[1] "data/penguins.csv"
```
R
```r
basename("data/penguins.csv")
```
```
[1] "penguins.csv"
```
R
```r
dirname("data/penguins.csv")
```
```
[1] "data"
```
R
```r
tools::file_ext("data/penguins.csv")
```
```
[1] "csv"
```

py
```python
os.path.join("data", "penguins.csv")
```
```
'data/penguins.csv'
```
py
```python
os.path.basename("data/penguins.csv")
```
```
'penguins.csv'
```
py
```python
os.path.dirname("data/penguins.csv")
```
```
'data'
```
py
```python
os.path.splitext("data/penguins.csv")
```
```
('data/penguins', '.csv')
```

Listing and checking

These calls list names inside `data`, without the `data/` prefix. Join each name to `"data"` before opening it, or use `full.names = TRUE` with R's `list.files()`.

R

```r
list.files("data", pattern = "\\.csv$")
```

```
[1] "d1.csv"
[2] "flights.csv"
[3] "mnist_digits.csv"
[4] "office_ratings.csv"
[5] "penguins.csv"
[6] "phone.csv"
```

R

```r
file.exists("data/penguins.csv")
```

```
[1] TRUE
```

py

```python
files = sorted(os.listdir("data"))
[f for f in files if f.endswith(".csv")]
```

```
['d1.csv',
 'flights.csv',
 'mnist_digits.csv',
 'office_ratings.csv',
 'penguins.csv',
 'phone.csv']
```

py

```python
os.path.exists("data/penguins.csv")
```

```
True
```

Creating and deleting

These examples create `out` and an empty placeholder file, then delete them. Creating an existing directory warns in R. Python raises an error unless `exist_ok=True`.

Both file-creation calls overwrite an existing file with an empty one. Deletion bypasses the trash, so check the path first. `os.rmdir()` removes only empty directories.

R
```r
dir.create("out")
file.create("out/a.png")
```

```
[1] TRUE
```

py
```python
os.makedirs("out", exist_ok=True)
open("out/a.png", "w").close()
```

R
```r
unlink("out", recursive = TRUE)
```

py
```python
os.remove("out/a.png")
os.rmdir("out")
```

R
```r
dir.exists("out")
```

```
[1] FALSE
```

py
```python
os.path.exists("out")
```

```
False
```

The `fs` package

The `fs` package gives R's file operations consistent names and behavior.

- Names group by task, with `path_`, `file_`, and `dir_` prefixes.
- Functions accept vectors of paths. Failed operations raise errors. Existence checks return `TRUE` or `FALSE`.
- Paths always use `/` as the separator, even on Windows.

```r
fs::dir_info("data", glob = "*.csv") |>
  dplyr::select(path, size, modification_time)
```

```
# A tibble: 6 × 3
  path                          size modification_time
  <fs::path>             <fs::bytes> <dttm>
1 data/d1.csv                 3.72K 2026-04-08 14:26:48
2 data/flights.csv           29.62M 2026-09-15 22:34:11
3 data/mnist_digits.csv     258.83K 2026-04-14 15:52:30
4 data/office_ratings.csv     7.02K 2022-08-29 10:47:54
5 data/penguins.csv          14.88K 2026-09-15 22:23:17
6 data/phone.csv                99 2022-08-29 10:47:54
```

Function reference

These functions serve similar purposes, but their arguments and return values differ. Names in the `fs` column assume the `fs::` prefix.

Task	Base R	`fs`	Python
Working directory	`getwd()`	`path_wd()`	`os.getcwd()`
Absolute path	`normalizePath()`	`path_abs()`	`os.path.abspath()`
Join paths	`file.path()`	`path()`	`os.path.join()`
File name	`basename()`	`path_file()`	`os.path.basename()`
Directory name	`dirname()`	`path_dir()`	`os.path.dirname()`
Extension	`tools::file_ext()`	`path_ext()`	`os.path.splitext()`
List files	`list.files()`	`dir_ls()`	`os.listdir()`
Check existence	`file.exists()`	`file_exists()`	`os.path.exists()`
Create directory	`dir.create()`	`dir_create()`	`os.makedirs()`
Delete	`unlink()`	`file_delete()`, `dir_delete()`	`os.remove()`, `os.rmdir()`
Copy	`file.copy()`	`file_copy()`	`shutil.copy()`
Move or rename	`file.rename()`	`file_move()`	`os.rename()`
Size, times, etc.	`file.info()`	`file_info()`	`os.stat()`